

SECURITY AUTOMATION

Full Week Study Guide — Theory + Practical

Beginner-friendly Python and cybersecurity automation notes covering Sections 1–9.

Fundamentals • Network Automation • Logs • Security Data • Monitoring • Workflows • Scheduling • AI
Assistance • Hands-on Tasks

Learning approach: Understand → Practice → Test → Review → Automate

Table of Contents

Section	Topic
1	Security Automation Fundamentals
2	Network Automation
3	Log Processing Automation
4	Security Data Processing
5	Monitoring Automation
6	Security Workflow Automation
7	Script Scheduling
8	AI-Assisted Automation
9	Hands-on Tasks
10	Combined Mini Project & Revision

Safety: Use network/security testing examples only on systems you own or are explicitly authorized to test.

1. Security Automation Fundamentals

Why Automation Matters

Security teams perform repetitive tasks such as reading logs, checking systems, collecting information, and creating reports. Automation uses scripts or tools to perform repeatable tasks quickly and consistently.

- Reduces repetitive manual work.
- Saves time for investigation.
- Makes repeatable tasks more consistent.
- Can reduce simple human errors.
- Allows checks to run on a schedule.

Security Operations Automation

Security operations automation means using software to perform security-related tasks with limited manual effort. Examples include log collection, event detection, monitoring, alerts, reports, and workflow steps.

Repetitive Task Reduction & Automation Opportunities

Good automation candidates are usually repetitive, rule-based, time-consuming, and clearly defined.

Task	Possible automation
Log checking	Search important events
System check	Read CPU, memory, disk
File integrity	Calculate and compare hashes
Reporting	Create text/JSON reports
Scheduled checks	Run automatically

SOC Automation Concepts

A Security Operations Center (SOC) monitors and investigates security events. Automation can collect data, filter events, correlate activity, create alerts, and prepare reports. Important decisions may still require human review.

Mental model: Detect → Analyze → Alert → Investigate → Respond → Report

2. Network Automation

Socket Programming Fundamentals

A socket is a software endpoint for network communication. Python's built-in socket module can create TCP/UDP connections.

```
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.settimeout(5)
sock.connect(("example.com", 80))

print("Connection successful")
sock.close()
```

Simple: AF_INET = IPv4. SOCK_STREAM = TCP. settimeout(5) limits waiting. connect() attempts a connection to a host and port. close() releases the socket.

Network Communication & Service Interaction

Network communication involves a source, destination, protocol, address, and port. A port identifies a service endpoint. A successful TCP connection does not by itself prove that an application protocol exchange succeeded.

Automated Service Check

```
import socket

host = "127.0.0.1"
port = 80

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.settimeout(2)

try:
    sock.connect((host, port))
    print("Service is UP")
except socket.timeout:
    print("Service check timed out")
except ConnectionRefusedError:
    print("Service is DOWN")
finally:
    sock.close()
```

Basic Scanning Concepts

Basic scanning can check authorized hosts or ports to identify reachable services. Start with simple checks and understand each connection attempt. Never scan systems without permission.

Mental model: Create socket → Set timeout → Connect → Check result → Close

3. Log Processing Automation

Log Collection

Log collection means obtaining records from applications, operating systems, devices, or security tools.

Log Parsing

Parsing turns raw log text into useful pieces such as timestamps, usernames, event types, and IP addresses.

Event Extraction

Event extraction identifies important events, such as failed login attempts or errors.

Event Correlation

Event correlation connects multiple events to find patterns. Several failed logins from the same IP can be investigated as a related pattern.

Automated Reporting

A report summarizes what the script found and can be displayed or saved.

```
from collections import Counter

failed_ips = []

with open("auth.log", "r") as file:
    for line in file:
        if "Failed login" in line:
            ip = line.split()[-1]
            failed_ips.append(ip)

counts = Counter(failed_ips)

print("SECURITY REPORT")
print("-----")

for ip, count in counts.items():
    print(ip, "→", count, "failed attempts")
    if count >= 3:
        print("WARNING: Suspicious activity")
```

Note: This example assumes the IP is the last field. Real logs can have different formats.

Mental model: Collect → Parse → Extract → Correlate → Report → Alert

4. Security Data Processing

Structured Data

Structured data is organized so a program can read it easily. Python dictionaries, CSV rows, and JSON objects are common examples.

```
event = {
    "user": "Alice",
    "ip": "192.168.1.10",
    "event": "login"
}
print(event["user"])
```

CSV Analysis

```
import csv

failed = 0

with open("security.csv", "r") as file:
    reader = csv.DictReader(file)
    for row in reader:
        if row["event"] == "failed_login":
            failed += 1

print("Failed logins:", failed)
```

JSON Analysis

```
import json

with open("security.json", "r") as file:
    events = json.load(file)

for event in events:
    print(event["username"])
    print(event["ip"])
    print(event["event"])
```

Event Categorization

```
event = "failed_login"

if event == "failed_login":
    print("Category: Authentication")
elif event == "file_access":
    print("Category: File Activity")
elif event == "malware_detected":
    print("Category: Malware")
else:
    print("Category: Other")
```

Alert Generation

```
event = "failed_login"
count = 6

if event == "failed_login" and count >= 5:
    severity = "HIGH"
elif event == "failed_login":
    severity = "LOW"
else:
    severity = "INFO"

print("Severity:", severity)
```

Mental model: Data → Read → Parse → Categorize → Count/Correlate → Rule → Alert → Report

5. Monitoring Automation

System Monitoring

Monitoring automation checks system conditions such as CPU, memory, disk usage, processes, and other measurable states.

Install psutil on Windows: **python -m pip install psutil**

```
import psutil

cpu = psutil.cpu_percent()
memory = psutil.virtual_memory().percent
disk = psutil.disk_usage("C:\\").percent

print("SYSTEM MONITOR")
print("-----")
print("CPU:", cpu, "%")
print("Memory:", memory, "%")
print("Disk:", disk, "%")

if cpu >= 80:
    print("ALERT: High CPU usage")
if memory >= 80:
    print("ALERT: High memory usage")
if disk >= 90:
    print("ALERT: High disk usage")
```

Service Monitoring

```
import socket

def check_service(host, port):
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.settimeout(2)
    try:
        sock.connect((host, port))
        return True
    except (socket.timeout, ConnectionRefusedError):
        return False
    finally:
        sock.close()

if check_service("127.0.0.1", 80):
    print("Service is UP")
else:
    print("Service is DOWN")
```

File Monitoring

```
import os

if os.path.exists("config.txt"):
    print("File exists")
else:
    print("File does not exist")

if os.path.exists("config.txt"):
    print("File size:", os.path.getsize("config.txt"), "bytes")
```

Change Detection with SHA-256

```
import hashlib

with open("config.txt", "rb") as file:
    data = file.read()

hash_value = hashlib.sha256(data).hexdigest()
print("SHA-256:", hash_value)
```

Security note: SHA-256 is useful for file integrity checking. It is not a password-storage algorithm by itself; use Argon2id, scrypt, or bcrypt for passwords.

Mental model: Monitor → Detect → Compare → Alert → Investigate

6. Security Workflow Automation

Ticket Creation

```
ticket = {
    "id": 1001,
    "title": "Multiple failed logins",
    "ip": "192.168.1.10",
    "severity": "High",
    "status": "Open"
```

```

}

print("Ticket ID:", ticket["id"])
print("Title:", ticket["title"])
print("Severity:", ticket["severity"])
print("Status:", ticket["status"])

```

Incident Workflow

An incident is a security event or group of events requiring investigation or response.

```

event = "failed_login"
count = 5

if event == "failed_login" and count >= 5:
    incident = {
        "type": "Multiple failed logins",
        "count": count,
        "severity": "High",
        "status": "Open"
    }
    print("Incident created")
    print(incident)

```

Notification

```

def send_notification(message):
    print("NOTIFICATION")
    print(message)

send_notification("Multiple failed logins detected")

```

Report Generation

```

incident = {
    "event": "Multiple failed logins",
    "ip": "192.168.1.10",
    "attempts": 5,
    "severity": "High",
    "status": "Open"
}

print("SECURITY REPORT")
print("-----")
for key, value in incident.items():
    print(key + ":", value)

```

Process Orchestration

```

def detect_event():
    return {"type": "failed_login", "ip": "192.168.1.10", "count": 5}

def create_ticket(event):
    print("Ticket created")

def send_notification(event):
    print("Notification sent")

def generate_report(event):
    print("Report generated")

event = detect_event()
create_ticket(event)
send_notification(event)
generate_report(event)

```

Workflow: Detect → Analyze → Ticket → Notify → Report. Keep human review for important decisions.

7. Script Scheduling

Scheduled Execution & Job Management

Scheduling automatically runs a script at a chosen time or interval. A job is a task that a scheduler runs. Job management defines what runs, when, and how often.

Execution Monitoring

```

from datetime import datetime

```

```
time = datetime.now()

with open("execution.log", "a") as file:
    file.write("Security check ran: " + str(time) + "\n")

print("Task completed")
```

Reliability Considerations

- Use try/except for expected failures.
- Write useful logs.
- Use timeouts for network operations.
- Use correct file paths.
- Protect passwords/API keys.
- Avoid duplicate jobs.
- Test manually before scheduling.

Windows Task Scheduler

On Windows, Task Scheduler can run Python scripts automatically. Find Python with:

```
where python
```

Use the Python executable as the program, the script path as an argument, and the project folder as the Start in directory. Test the command manually first.

Scheduled Security Check

```
import psutil
from datetime import datetime

try:
    cpu = psutil.cpu_percent()
    memory = psutil.virtual_memory().percent
    disk = psutil.disk_usage("C:\\").percent

    print("CPU:", cpu, "%")
    print("Memory:", memory, "%")
    print("Disk:", disk, "%")

    with open("execution.log", "a") as file:
        file.write(str(datetime.now()) + " - Check completed\n")

    if cpu >= 80:
        print("ALERT: High CPU usage")
    if memory >= 80:
        print("ALERT: High memory usage")
    if disk >= 90:
        print("ALERT: High disk usage")

except Exception as error:
    print("ERROR:", error)
```

Mental model: Schedule → Run → Monitor → Detect → Alert → Log → Repeat

8. AI-Assisted Automation

Use AI For

- Script generation from clear requirements.
- Log parsing ideas and extraction rules.
- Workflow design before coding.
- Automation improvements such as error handling and readability.
- Generating test cases and explanations.

Validate AI Outputs

Validation	Question
Review automation logic	Does the code match the requirement?
Verify workflow behavior	Does it work with normal, boundary, and invalid input?
Assess security	Can input cause unsafe behavior? Are secrets exposed?
Test	Do actual results match expected results?

AI workflow: Generate → Understand → Review → Test → Security-check → Verify → Use

9. Apply Through Hands-on Tasks

Task 1 — Build a Log Analysis Script

Read security.log, count failed and successful logins, and print a summary.

```
failed = 0
successful = 0

with open("security.log", "r") as file:
    for line in file:
        if "FAILED" in line:
            failed += 1
        elif "SUCCESS" in line:
            successful += 1

print("Failed logins:", failed)
print("Successful logins:", successful)
```

Task 2 — Create a Monitoring Tool

Monitor CPU, memory, and disk usage with psutil. Print an alert when a threshold is reached.

Task 3 — Develop an Automation Workflow

Detect an event, create a ticket object, send a notification, and generate a report.

Task 4 — Generate Security Reports

Save event details such as IP, attempts, severity, and status to a text or JSON report.

Task 5 — Build an Incident-Support Utility

Create a small utility that collects incident information and presents it clearly for human investigation.

Practice progression: Build one task → Test → Create functions → Combine tasks → Add logging → Schedule it.

10. Combined Mini Project & Revision

Suggested Project Structure

```
security_automation/
├── main.py
├── log_analyzer.py
├── monitor.py
├── incident.py
├── report.py
├── security.log
├── security_report.txt
└── execution.log
```

Combined Workflow

```
Detect Event
↓
Analyze Event
↓
Create Ticket
```

↓
Send Notification
↓
Generate Report
↓
Human Investigation

Skills Covered

Skill	Used for
Variables / dictionaries	Events and tickets
Conditions	Rules and alerts
Loops	Log processing
Functions	Reusable checks
Files	Logs and reports
CSV / JSON	Security data
Exceptions	Reliable scripts
socket / os / hashlib / psutil	Security utilities
Scheduling	Automatic execution
AI assistance	Generation, design, review

Final Checklist

- Explain why security automation is useful.
- Create a basic TCP service check.
- Read and parse a log file.
- Explain event correlation.
- Process CSV and JSON.
- Monitor CPU, memory, and disk.
- Use SHA-256 for file integrity.
- Create a basic incident workflow.
- Schedule a Python script on Windows.
- Use AI to assist with automation.
- Review and test AI-generated code.
- Build a small security automation project.

Final Security Reminder

Use these techniques only on systems, networks, files, and accounts you own or are explicitly authorized to test. Automation should make work safer and more consistent; important security decisions should have appropriate human review.